Image transformations with PyTorch

Operator manual | lorem-warped | Version 1.4

This manual uses selected writing principles from ASD-STE100 Issue 9. It is not certified ASD-STE100 documentation. Formal conformity needs a controlled-dictionary and rule review.

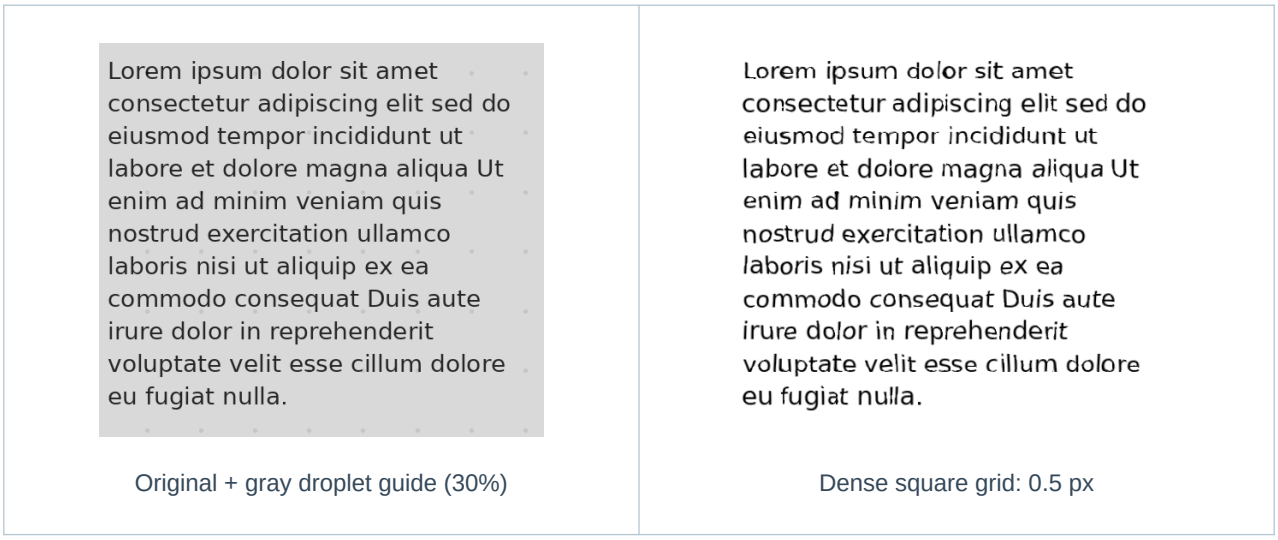
1. Package introduction

lorem-warped applies independent PyTorch image transformations to a single RGB image. It includes droplet layouts, page waves, a six-control sine lens, a gentle pinch, a refractive-index slab, and a Python-only custom callback. Input and output are CHW tensors; the CLI writes PNG files.

Package structure: src/lorem_warped/ holds the library and CLI. examples/ holds a runnable script. docs/build_manual.py rebuilds this PDF and its real examples. tests/ checks the interfaces. original.png is the sample input. No command in this manual requires you to replace that input.

1.1 Distortion overview

A distortion moves source pixels to new positions. The left image is the unwarped original with its actual droplet regions shaded gray at 30% opacity. The right image uses a dense square-grid droplet warp. The guide is display-only: it is not drawn on the transform input or output.

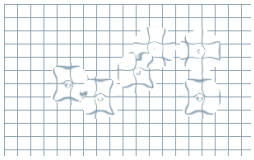
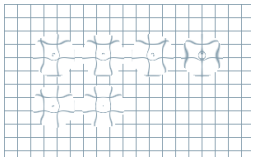
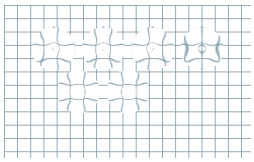
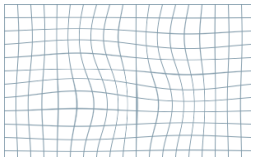
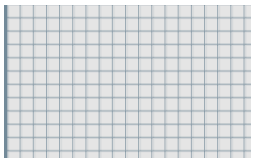
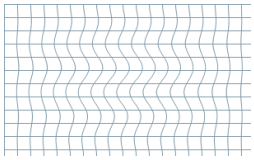
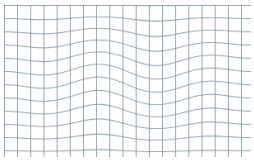
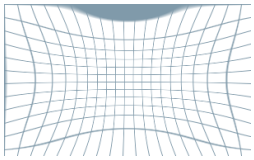
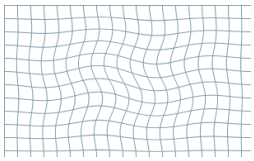


For this input, request 392 square-grid centres, a radius of 70 pixels, and an intensity of 0.5 pixels. This low intensity keeps the dense example legible. More centres make the grid denser. Radius sets each droplet's size. No droplet example in this manual exceeds 5 pixels of intensity.

Choose a transform: droplets give local circles; a sine lens and other page warps move larger page regions; refraction makes two translated views. Choose random, square, or triangle (hexagonal) centres for droplets. Set centre count, radius, intensity, seed, and profile. The other modes do not use centre distribution.

2. Distortion types

Read this tree from left to right. The middle column applies each actual transform to the same dense, text-free square grid. The right column applies it to the same short paragraph. Grid lines move with the transform; they are not droplet centres. Droplet layouts use 3 px intensity, and a uniform slab shifts lines without bending them.

Type	Distorted square grid	Distorted text
`|-- droplets` `    |-- random`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|   |-- square grid`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|   `-- triangular grid`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Sine lens`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Refractive-index slab`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Horizontal wave`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Vertical wave`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Gentle pinch`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Custom wave`		Lorem ipsum dolor sit amet consectetur adipiscing elit

The slab view is a uniform translation of two polarization modes. Its preview does not imply a spatial lens. A custom callback has no CLI command.

3. Install and prepare

Install Python 3.10 or later. Open a shell in the repository root. Make a virtual environment. Install the project with its manual tools.

a. CLI

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -e '.[dev]'
python -m lorem_warped --help
```

On Windows PowerShell, activate .venv\Scripts\Activate.ps1. Use the PyTorch installation selector if you need a specific GPU build.

Use original.png as the input. Do not overwrite it. The CLI uses a generated page if you omit --input. Python images use float CHW tensors from 0 to 1.

4. Script setup

In the script examples, read the image one time. Use this function to save each output tensor as a PNG. Each transform starts from source, not from the output of another transform.

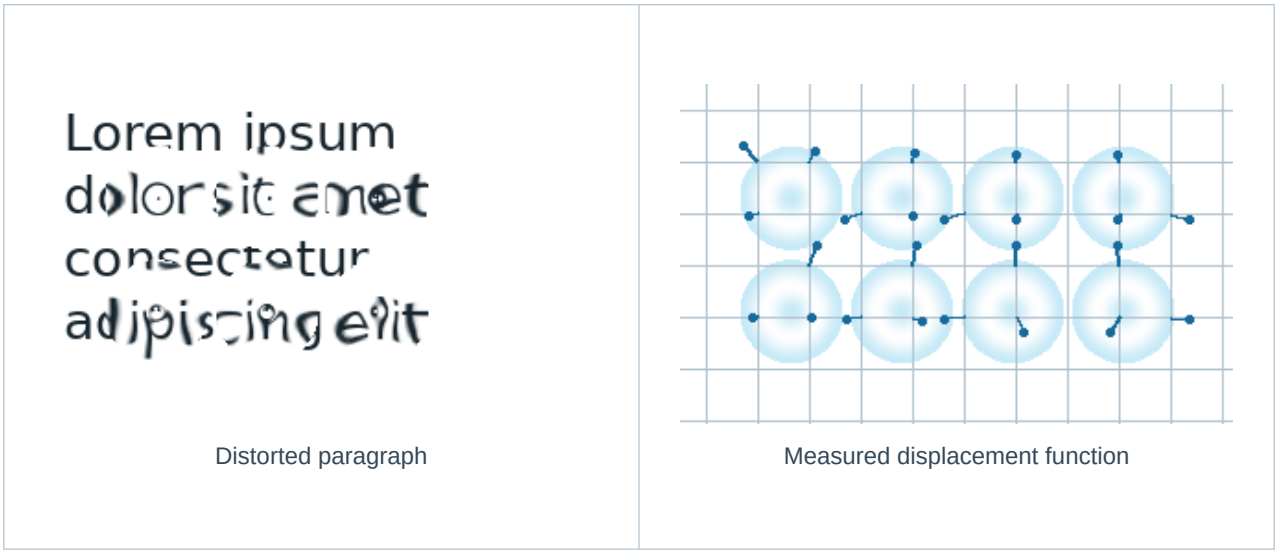
a. in script

```
from PIL import Image
from lorem_warped import (
    CentreDistribution, DistortionConfig,
    WarpTransform, image_to_tensor)
source = image_to_tensor(Image.open('original.png'))
def save(path, result):
    pixels = result.mul(255).round().byte()
    pixels = pixels.permute(1, 2, 0).cpu().numpy()
    Image.fromarray(pixels).save(path)
```

A custom function also needs math and torch imports. The custom code example includes them. The CLI runs on CPU. Move the tensor to a supported GPU in your script if necessary.

5. How droplet transformation works

Each centre has a radius. The cosine profile gives a radial displacement that fades to zero at the circle edge. Overlapping fields add before the image is sampled once. Pixel intensity multiplies the field. The right-hand plot shows the measured displacement vectors and shaded influence on a square reference grid.



Distorted paragraph

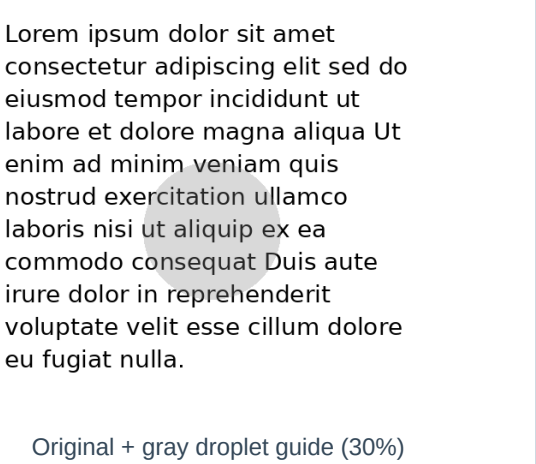
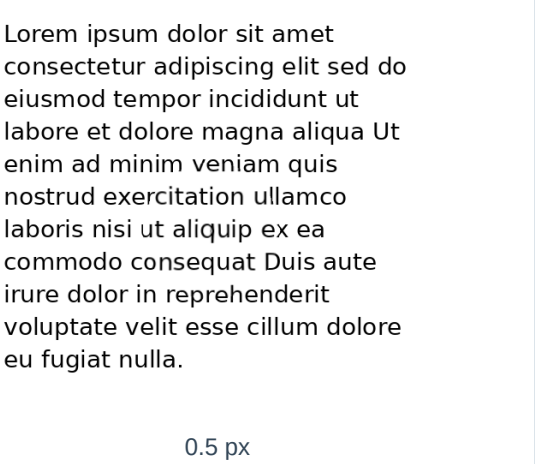
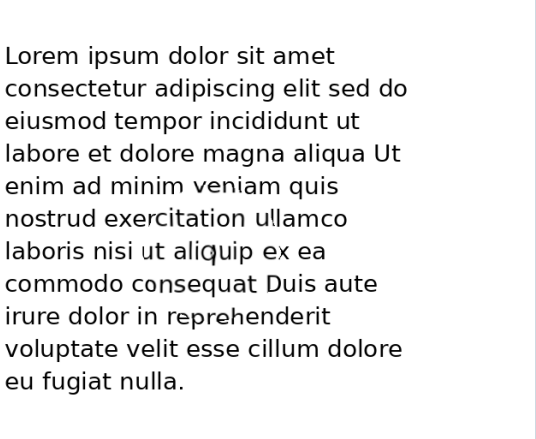
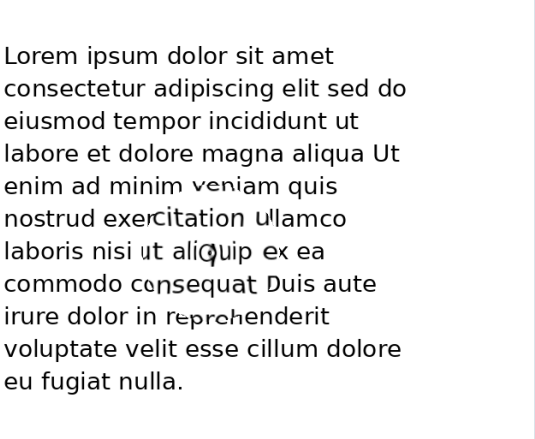
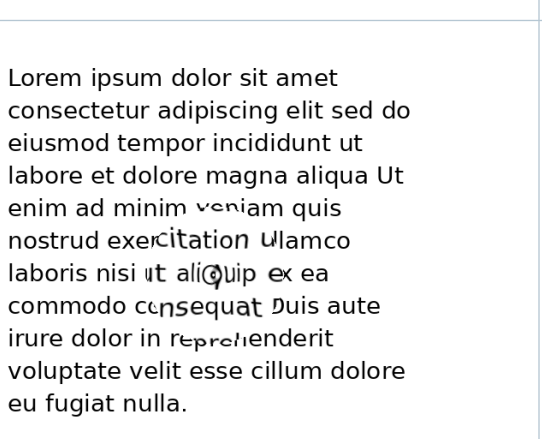
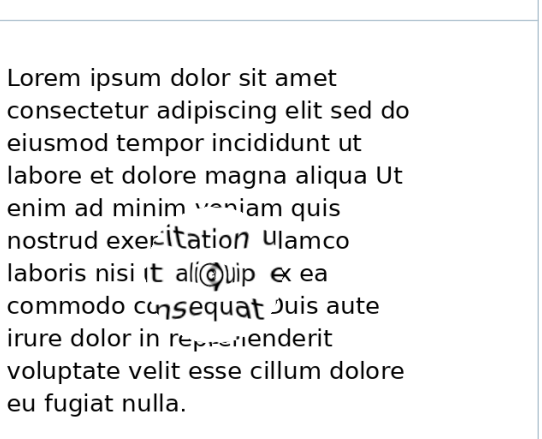
Measured displacement function

The left panel uses square; 8 centres; radius 28 px; intensity 2.5 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

5.1 Droplet transformation: strength examples

A droplet warp moves pixels around circular centres. Choose random, square, or triangle (hexagonal) centres. Radius sets each circle size. Intensity sets the movement in pixels. The seed fixes random centres. The cosine profile permits overlaps.

The first panel is the unwarped original with display-only gray droplet guides at 30% opacity. The other panels warp the unmarked source. Only pixel intensity changes; no example exceeds 5 pixels.

 Original + gray droplet guide (30%)	 0.5 px
 1 px	 2 px
 3 px	 5 px

5.2 Run droplet transformation

a. CLI

```
python -m lorem_warped --input original.png \
  --output droplet.png --centres 3 --distribution random \
  --radius 80 --intensity 5 --seed 7 --profile cosine
```

b. in script

```
transform = WarpTransform(mode='droplet',
    config=DistortionConfig(centre_count=3, radius=80,
                            intensity=5, seed=7))
save('droplet.png', transform(source))
```

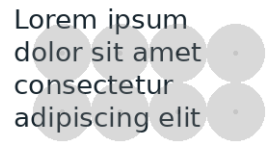
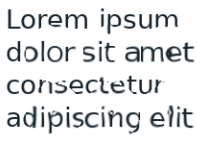
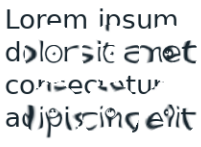
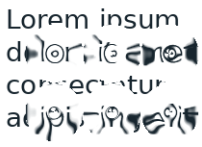
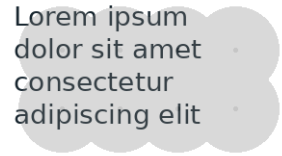
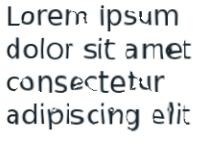
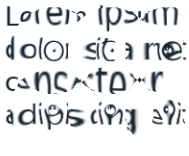
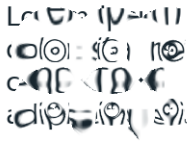
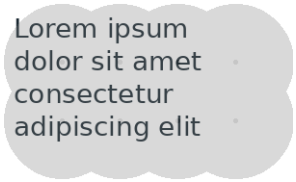
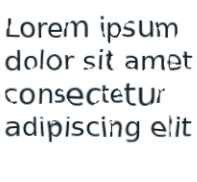
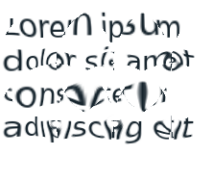
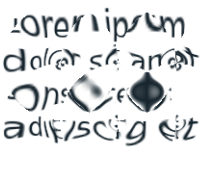
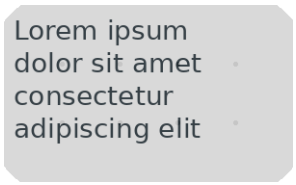
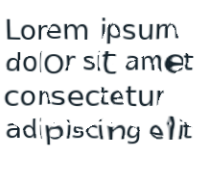
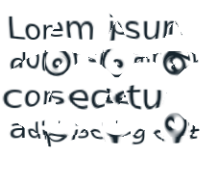
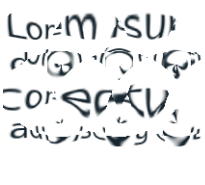
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

5.3 Two-variable legibility study

Rows change radius (px); columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

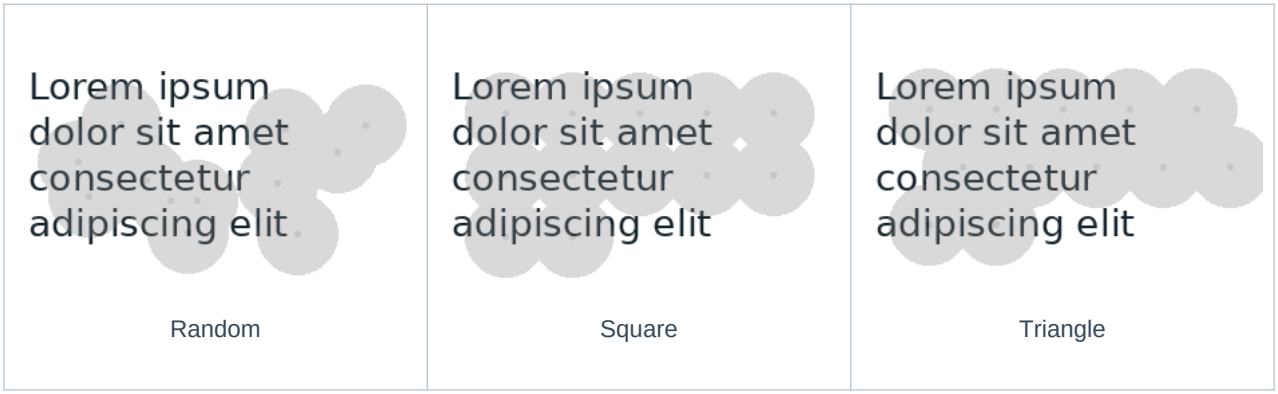
Zero-warp controls show gray droplet regions at 30% opacity. Other cells use actual droplet warps at no more than 5 pixels of intensity.

Keep all other transform settings fixed. The first column includes a zero-intensity control where applicable.

 30 | 0	 30 | 1	 30 | 3	 30 | 5
 45 | 0	 45 | 1	 45 | 3	 45 | 5
 60 | 0	 60 | 1	 60 | 3	 60 | 5
 75 | 0	 75 | 1	 75 | 3	 75 | 5

5.4 Shaded droplet regions

Each gray circle has 30% opacity and shows one droplet's radius. Each gray dot marks a centre. The same original text lies under every shaded map. These are centre-placement guides, not warped output images.



Random uses the seed to place centres. Square uses aligned rows and columns. Triangle offsets alternate rows to make a hexagonal neighbour pattern. The shaded regions can overlap with a cosine profile; parabolic profiles need nonoverlapping centres.

5.5 Droplet arrangements

The distribution controls centre positions. Random selects seeded positions. Square places centres in aligned rows and columns. Triangle offsets alternate rows and makes a hexagonal neighbour pattern. In the CLI, select the hexagonal pattern with --distribution triangle, not hex.

Compare the same text fragment. Each row pairs an unwarped original with its actual droplet regions shaded gray at 30% opacity (left) and the warped output (right). Request 71 centres for each layout. Keep radius 50 pixels, intensity 3 pixels, seed 7, and the cosine profile fixed. Only the distribution changes. The overlays are display-only. A shifted triangle row can have fewer centres near an edge.



5.6 Select a droplet distribution

Use one of these three names. Keep all other controls fixed to compare the centre layouts. The CLI commands below use the count and radius shown on the preceding page.

random. CLI

```
python -m lorem_warped --input original.png \
  --output droplets_random.png --centres 71 \
  --distribution random --radius 50 \
  --intensity 3 --seed 7 --profile cosine
```

square. CLI

```
python -m lorem_warped --input original.png \
  --output droplets_square.png --centres 71 \
  --distribution square --radius 50 \
  --intensity 3 --seed 7 --profile cosine
```

triangle. CLI

```
python -m lorem_warped --input original.png \
  --output droplets_triangle.png --centres 71 \
  --distribution triangle --radius 50 \
  --intensity 3 --seed 7 --profile cosine
```

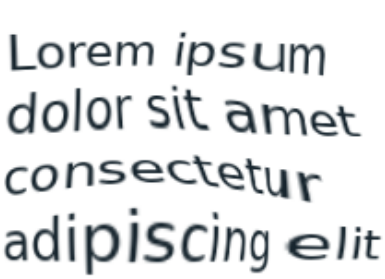
d. in script

```
for name in ('random', 'square', 'triangle'):
    config = DistortionConfig(centre_count=71,
        distribution=CentreDistribution(name), radius=50,
        intensity=3, seed=7, profile='cosine')
    save('droplets_' + name + '.png',
         WarpTransform(mode='droplet', config=config)(source))
```

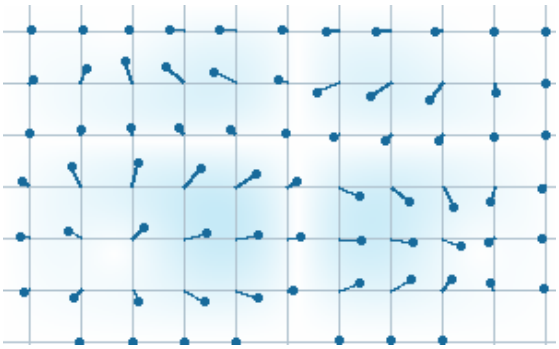
The script uses the imports and save function from section 4. A fixed centre count controls square and triangle density. A fixed seed repeats the random pattern. The cosine profile permits overlaps. A parabolic profile requires disjoint circles; --nonoverlapping-page makes disjoint page-wide layouts and overrides --centres.

6. How six-parameter sine lens works

The sine lens combines a two-axis sinusoidal envelope with local cell-centre directions. X and Y frequencies set the cell counts. Phase moves the lobes; axis gains set the X and Y movement. Pixel intensity multiplies the field.

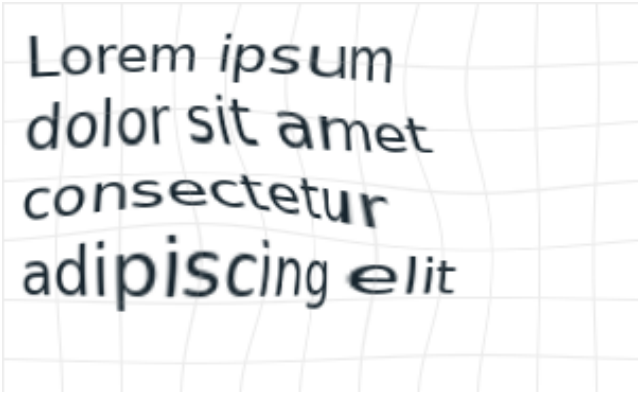


Distorted paragraph



Measured displacement function

The left panel uses X/Y frequency 1.5/1.5; intensity 80 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

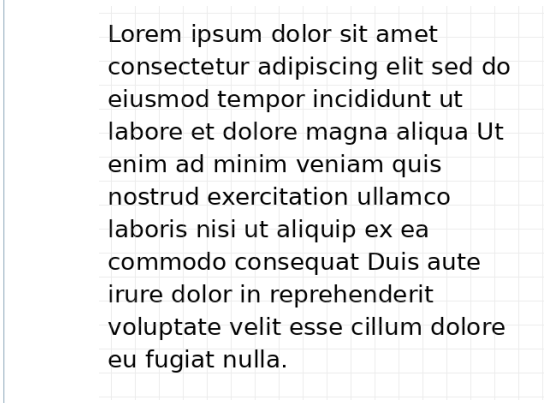


The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

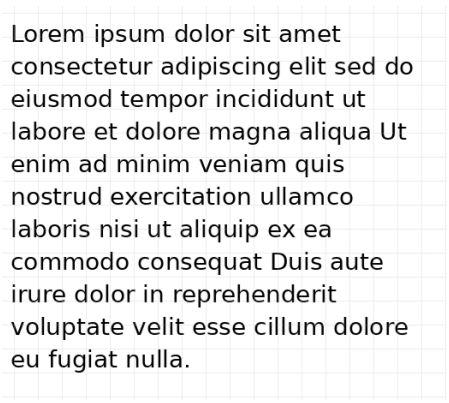
6.1 Six-parameter sine lens: strength examples

A sine lens moves the page in two axes. X and Y frequencies, phases, and gains give six shape parameters. The intensity multiplies the displacement in pixels. Keep the six parameters fixed in this comparison.

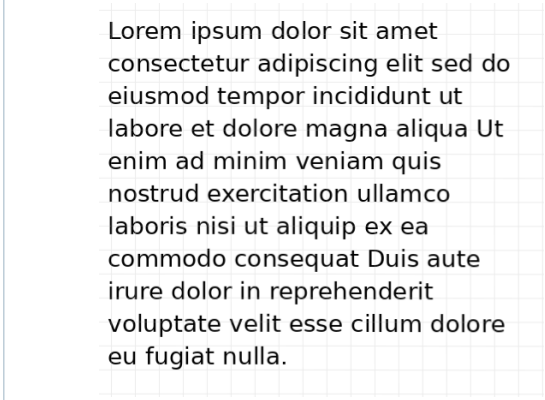
The first panel shows a straight, dense square grid over the original. Each output uses one sine_lens transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



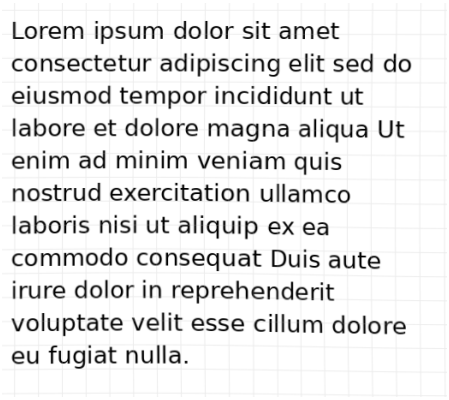
Original + straight square grid



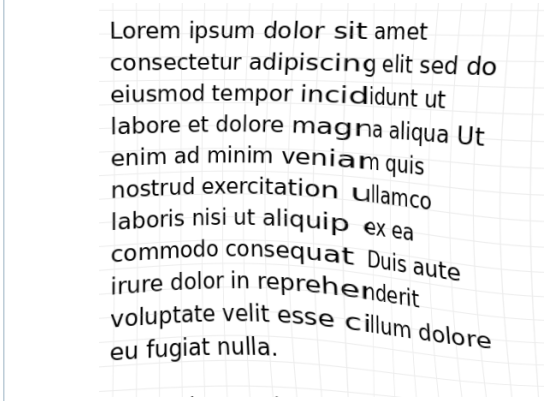
0.5 px



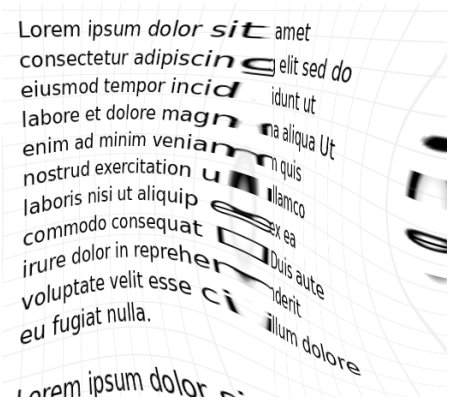
4 px



32 px



256 px



1024 px

6.2 Run six-parameter sine lens

Set the six lens parameters to X/Y frequencies 2.5/1.5, phases 0.1/0.2 cycles, and gains 1.2/0.8. Use 32 pixels for this example. A frequency must be positive.

a. CLI

```
python -m lorem_warped --input original.png \
  --output sine_lens.png --page-warp sine_lens \
  --intensity 32 --lens-frequency-x 2.5 \
  --lens-frequency-y 1.5 --lens-phase-x 0.1 \
  --lens-phase-y 0.2 --lens-amplitude-x 1.2 \
  --lens-amplitude-y 0.8
```

b. in script

```
transform = WarpTransform(mode='sine_lens', intensity=32,
    lens_options=dict(frequency_x=2.5, frequency_y=1.5,
        phase_x=0.1, phase_y=0.2,
        amplitude_x=1.2, amplitude_y=0.8))
save('sine_lens.png', transform(source))
```

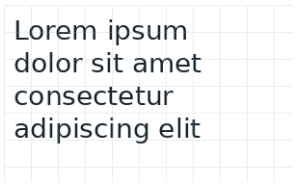
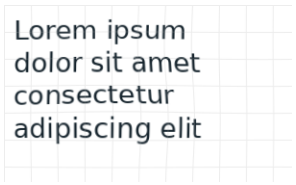
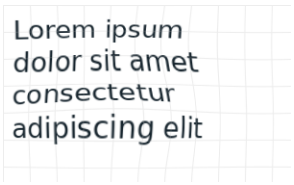
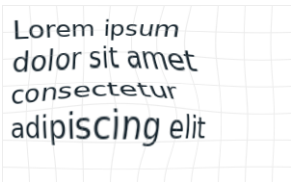
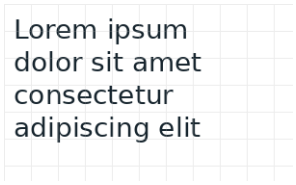
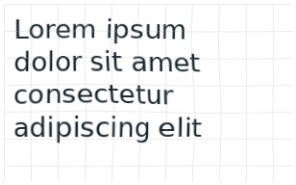
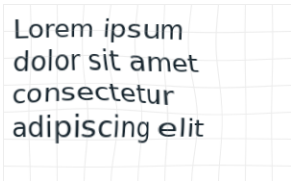
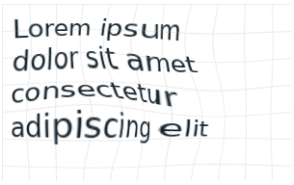
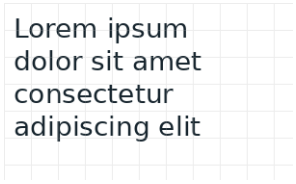
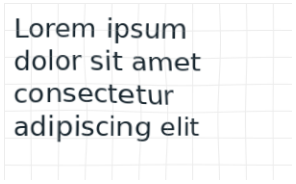
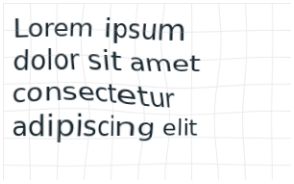
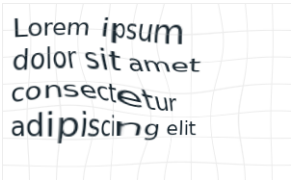
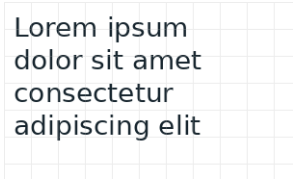
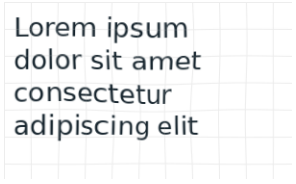
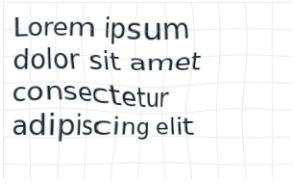
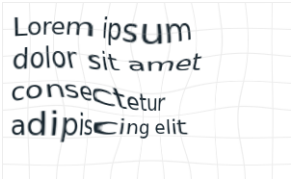
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

6.3 Two-variable legibility study

Rows change X frequency; columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

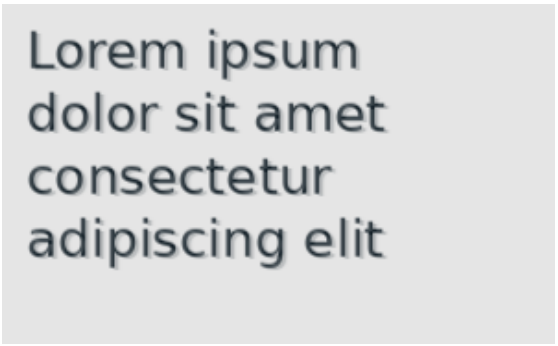
Every cell starts with the same dense square reference grid. The transform warps the grid with the text; zero-intensity cells keep straight lines. These lines do not represent droplet centres.

Keep all other transform settings fixed. The first column includes a zero-intensity control where applicable.

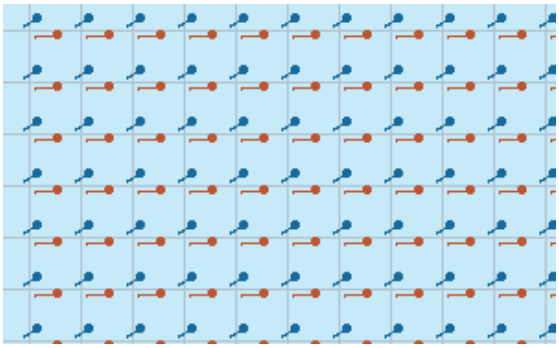
 1 | 0	 1 | 15	 1 | 40	 1 | 80
 1.5 | 0	 1.5 | 15	 1.5 | 40	 1.5 | 80
 2 | 0	 2 | 15	 2 | 40	 2 | 80
 2.5 | 0	 2.5 | 15	 2.5 | 40	 2.5 | 80

7. How refractive-index transformation works

The uniform slab yields two distinct constant translations: one for each polarization mode. Thickness changes their offsets. Polarization changes their weights. The right-hand plot shows both vectors, not a varying lens field. Displacement is converted from millimetres with the page dimensions.

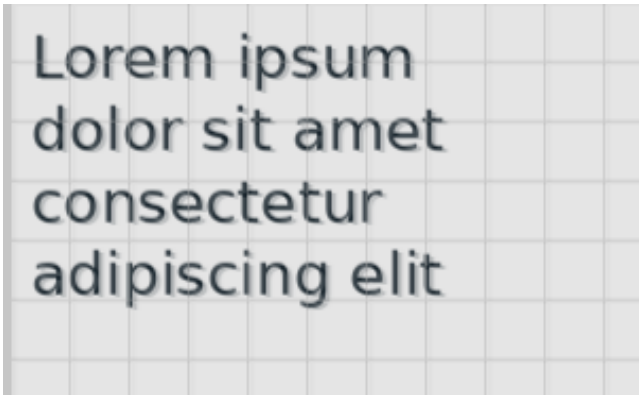


Distorted paragraph



Measured displacement function

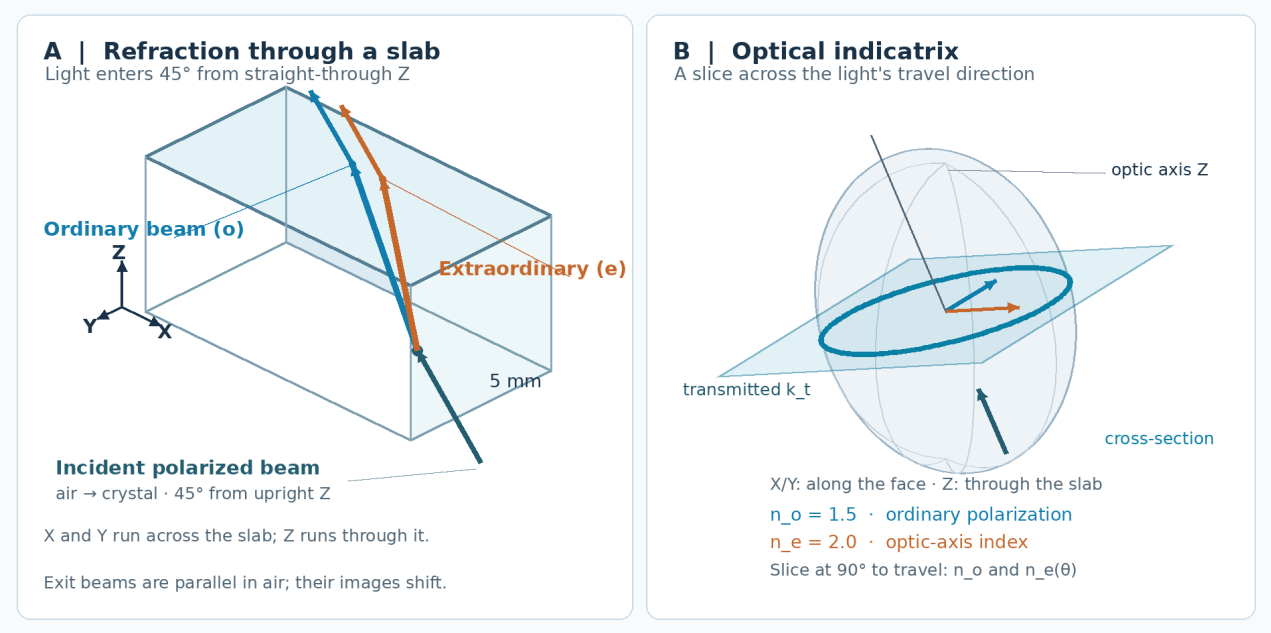
The left panel uses thickness 5 mm; polarization 30 degrees. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.



The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

7.0 Ray geometry and optical indicatrix

X and Y run along the slab's two flat edges; Z points through its thickness. The incoming beam tilts 45 degrees away from straight-through Z toward negative X, with no travel along Y: halfway between straight through and sideways. In this uniaxial example, the blue Ordinary beam and orange Extraordinary beam follow different paths, then leave parallel. Their paths come from the slab solver; the picture is not to scale.



At right, imagine slicing the optical indicatrix (an egg-shaped index map) across the direction the light travels. This cyan cross-section is at 90 degrees to the transmitted wave normal; its colored arrows mark two ways the light can wiggle (polarization). Here n_x = n_y = 1.5, n_z = 2.0, thickness = 5 mm, and incident direction = (-1, 0, 1). The separate n_x-by-n_y studies can be biaxial, so they use X/Y eigenmode labels instead of ordinary/extraordinary.

Schematic terminology: Tulane University, Uniaxial Minerals, Uniaxial Indicatrix, Optic Sign, and Ray Path. The image uses this manual's optical solver for the ray positions; polarization weights at oblique incidence remain approximate.

Optical indicatrix reference: https://www2.tulane.edu/~sanelson/eens211/uniaxial_minerals.htm

7.1 Refractive-index transformation: strength examples

A uniform anisotropic slab makes two translated polarization views. Thickness sets the translation in millimetres. The refractive indices and physical page size stay fixed. Do not use pixel intensity for this physical model.

The first panel shows a straight, dense square grid over the original. Each output uses one refractive_index transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only mm thickness changes. Strong settings may reduce legibility.



7.2 Run refractive-index transformation

Thickness and page dimensions use millimetres. The angle uses degrees. The slab gives constant translations, not a spatially varying lens. The polarization weights are approximate at oblique incidence.

a. CLI

```
python -m lorem_warped --input original.png \
  --output refraction.png --page-warp refractive_index \
  --refraction-thickness-mm 5 --refraction-n-x 1.3 \
  --refraction-n-y 2.5 --refraction-n-z 1.3 \
  --refraction-polarization-deg 45 \
  --refraction-direction -1 1 1 \
  --page-width-mm 210 --page-height-mm 297
```

b. in script

```
transform = WarpTransform(mode='refractive_index',
    refraction_options=dict(thickness_mm=5, n_x=1.3,
        n_y=2.5, n_z=1.3, polarization_deg=45,
        direction_xyz=(-1, 1, 1),
        page_width_mm=210, page_height_mm=297))
save('refraction.png', transform(source))
```

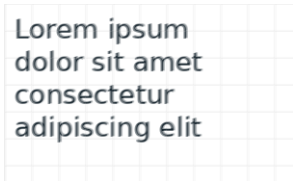
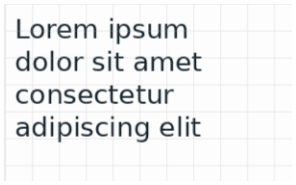
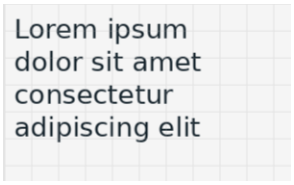
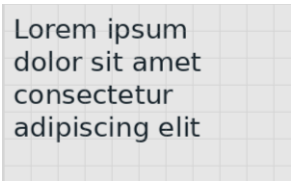
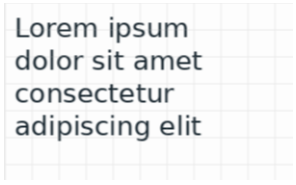
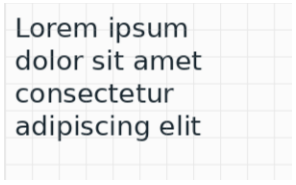
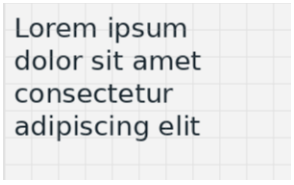
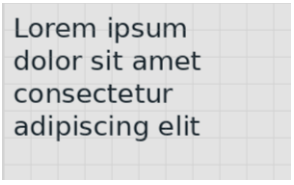
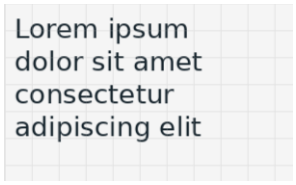
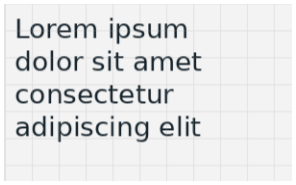
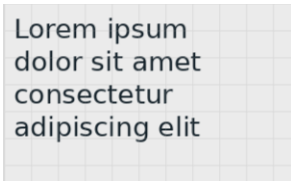
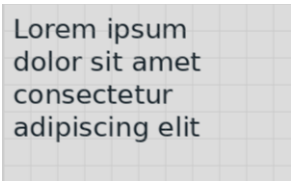
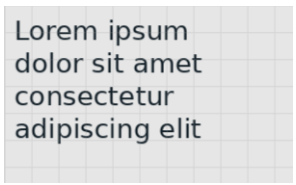
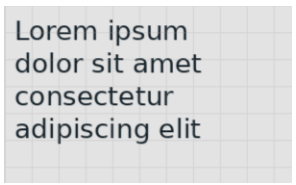
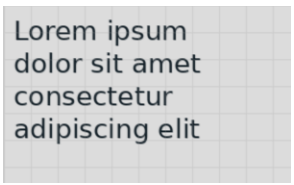
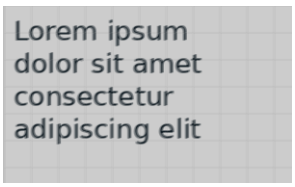
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

7.3 Two-variable legibility study: 2 mm

Rows change n_x; columns change n_y. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

Every cell starts with the same dense square reference grid. The uniform slab translates the grid with the text; grid lines stay straight, though their contrast can change. These lines do not represent droplet centres.

Keep slab thickness at 2 mm and polarization at 45 degrees. Change n_x down the rows and n_y across the columns: 1, 1.2, 1.5, 2.0. The indices are dimensionless; n_z remains 1.3. A uniform slab produces translated views, not a curved lens. No artistic pixel intensity is used.

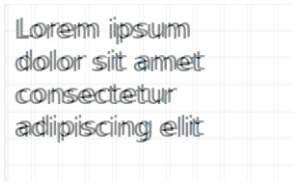
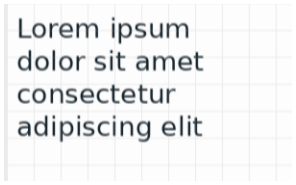
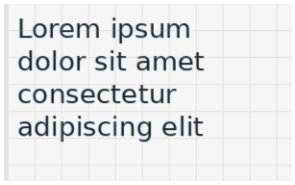
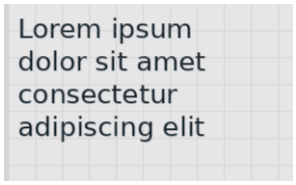
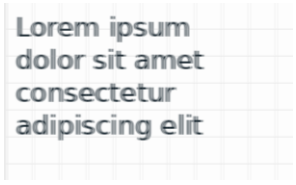
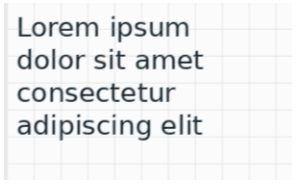
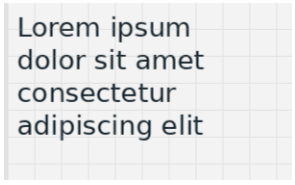
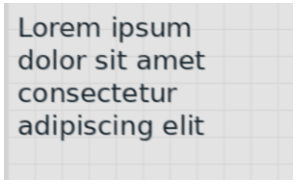
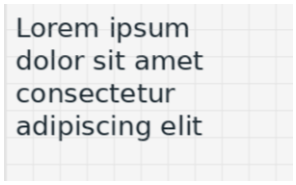
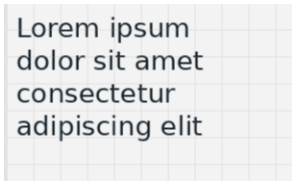
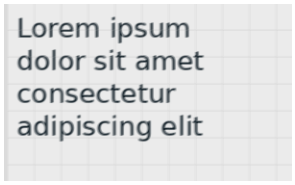
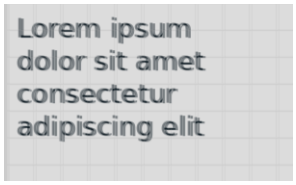
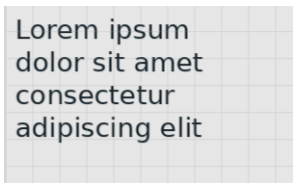
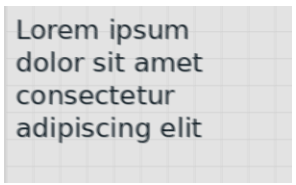
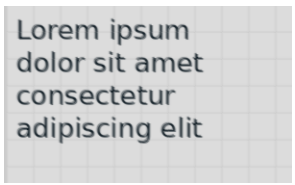
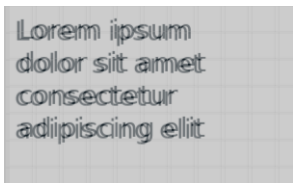
 1 | 1	 1 | 1.2	 1 | 1.5	 1 | 2
 1.2 | 1	 1.2 | 1.2	 1.2 | 1.5	 1.2 | 2
 1.5 | 1	 1.5 | 1.2	 1.5 | 1.5	 1.5 | 2
 2 | 1	 2 | 1.2	 2 | 1.5	 2 | 2

7.3 Two-variable legibility study: 5 mm

Rows change n_x; columns change n_y. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

Every cell starts with the same dense square reference grid. The uniform slab translates the grid with the text; grid lines stay straight, though their contrast can change. These lines do not represent droplet centres.

Keep slab thickness at 5 mm and polarization at 45 degrees. Change n_x down the rows and n_y across the columns: 1, 1.2, 1.5, 2.0. The indices are dimensionless; n_z remains 1.3. A uniform slab produces translated views, not a curved lens. No artistic pixel intensity is used.

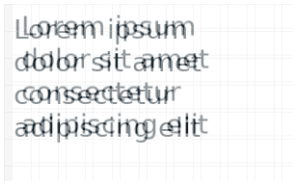
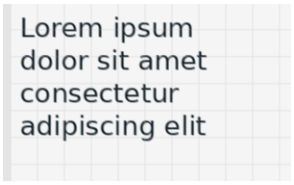
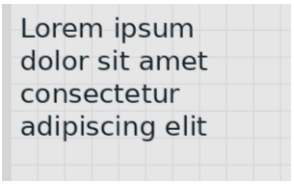
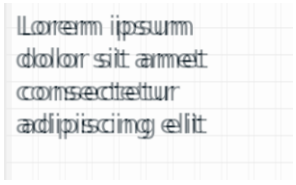
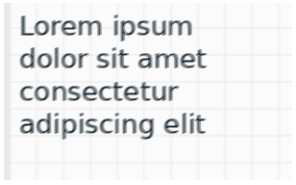
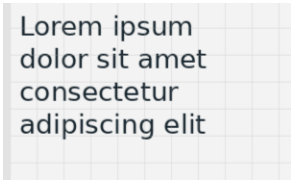
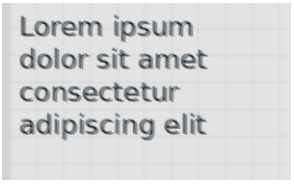
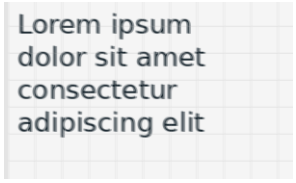
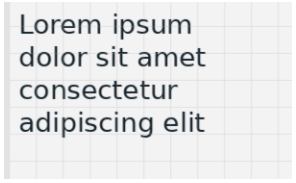
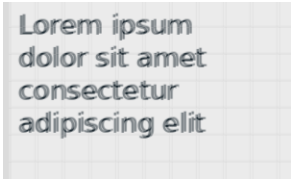
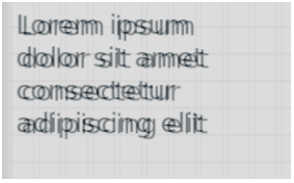
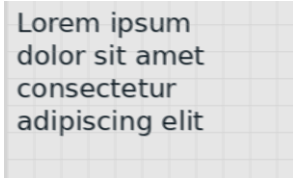
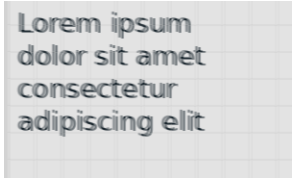
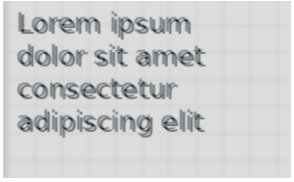
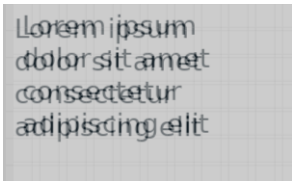
 1 | 1	 1 | 1.2	 1 | 1.5	 1 | 2
 1.2 | 1	 1.2 | 1.2	 1.2 | 1.5	 1.2 | 2
 1.5 | 1	 1.5 | 1.2	 1.5 | 1.5	 1.5 | 2
 2 | 1	 2 | 1.2	 2 | 1.5	 2 | 2

7.3 Two-variable legibility study: 10 mm

Rows change n_x; columns change n_y. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

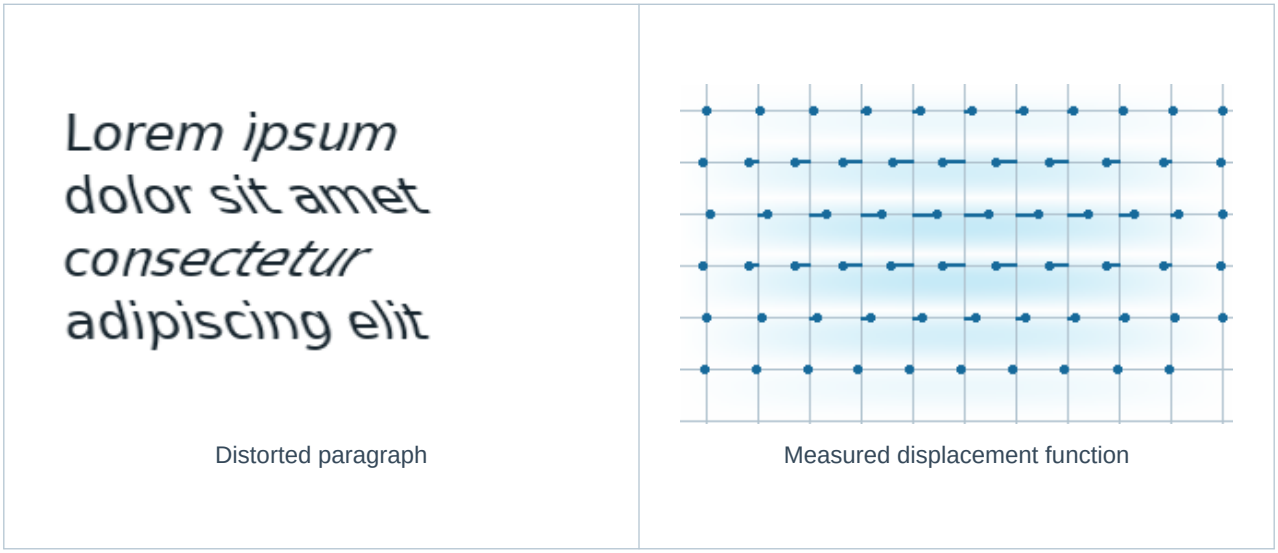
Every cell starts with the same dense square reference grid. The uniform slab translates the grid with the text; grid lines stay straight, though their contrast can change. These lines do not represent droplet centres.

Keep slab thickness at 10 mm and polarization at 45 degrees. Change n_x down the rows and n_y across the columns: 1, 1.2, 1.5, 2.0. The indices are dimensionless; n_z remains 1.3. A uniform slab produces translated views, not a curved lens. No artistic pixel intensity is used.

 1 | 1	 1 | 1.2	 1 | 1.5	 1 | 2
 1.2 | 1	 1.2 | 1.2	 1.2 | 1.5	 1.2 | 2
 1.5 | 1	 1.5 | 1.2	 1.5 | 1.5	 1.5 | 2
 2 | 1	 2 | 1.2	 2 | 1.5	 2 | 2

8. How horizontal wave works

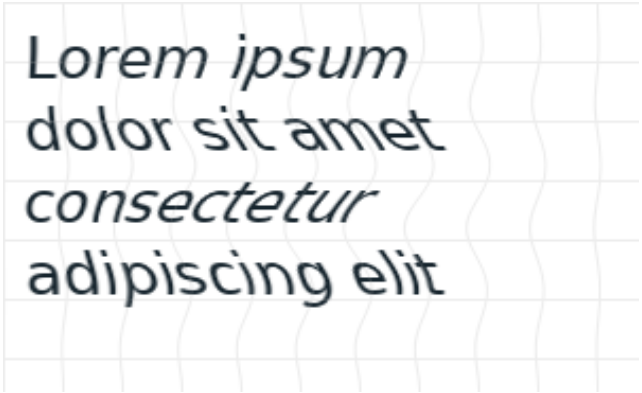
A fixed three-cycle wave moves text horizontally. The function has a smooth edge envelope, so page edges stay fixed. Pixel intensity multiplies the horizontal displacement.



Distorted paragraph

Measured displacement function

The left panel uses intensity 8 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.



The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

8.1 Horizontal wave: strength examples

A horizontal wave moves text left and right in broad bands. The page edges stay in place. The intensity sets the movement in pixels.

The first panel shows a straight, dense square grid over the original. Each output uses one horizontal_wave transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



8.2 Run horizontal wave

a. CLI

```
python -m lorem_warped --input original.png \
  --output horizontal_wave.png --page-warp horizontal_wave \
  --intensity 60
```

b. in script

```
transform = WarpTransform(mode='horizontal_wave', intensity=60)
save('horizontal_wave.png', transform(source))
```

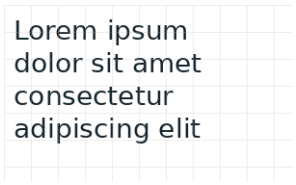
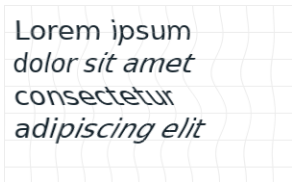
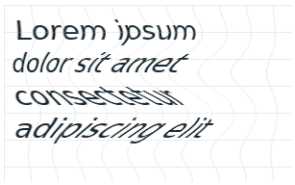
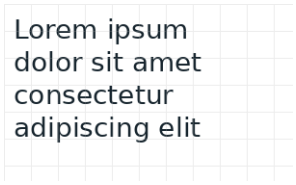
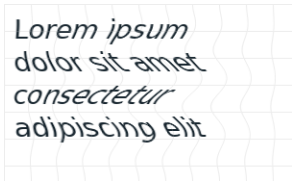
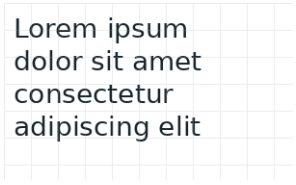
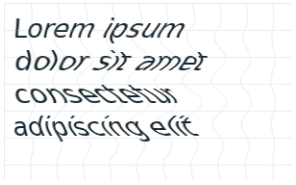
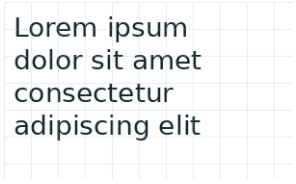
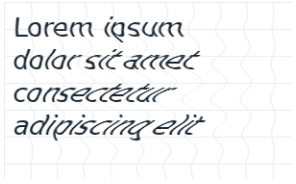
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

8.3 Two-variable legibility study

Rows change horizontal wave cycles; columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

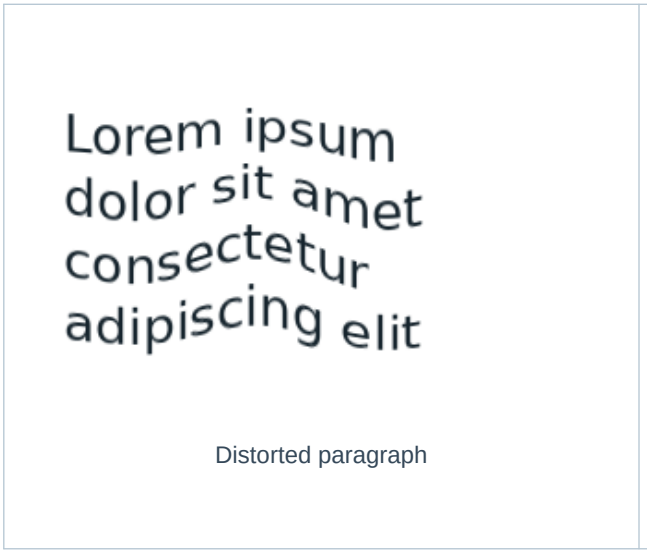
Every cell starts with the same dense square reference grid. The transform warps the grid with the text; zero-intensity cells keep straight lines. These lines do not represent droplet centres.

The built-in preset exposes only intensity. This study uses a Python callback with an adjustable wave-cycle count or pinch-envelope power; it is a parameterized variant, not an extra preset CLI flag. The input text itself never changes.

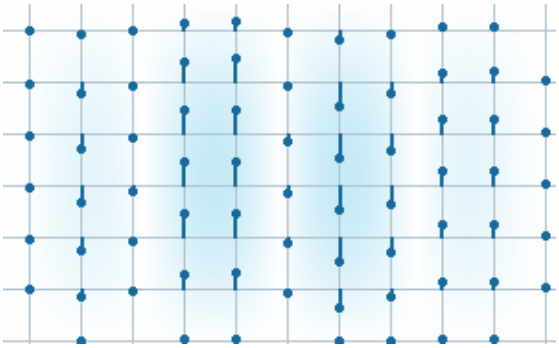
 2 | 0	 2 | 12	 2 | 28	 2 | 50
 3 | 0	 3 | 12	 3 | 28	 3 | 50
 4 | 0	 4 | 12	 4 | 28	 4 | 50
 5 | 0	 5 | 12	 5 | 28	 5 | 50

9. How vertical wave works

A fixed two-cycle wave moves text vertically. The function has a smooth edge envelope, so page edges stay fixed. Pixel intensity multiplies the vertical displacement.

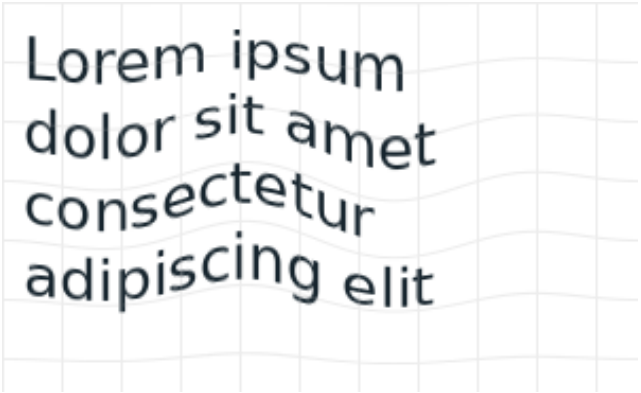


Distorted paragraph



Measured displacement function

The left panel uses intensity 10 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.



The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

9.1 Vertical wave: strength examples

A vertical wave moves baselines up and down across the page. The page edges stay in place. The intensity sets the movement in pixels.

The first panel shows a straight, dense square grid over the original. Each output uses one vertical_wave transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



9.2 Run vertical wave

a. CLI

```
python -m lorem_warped --input original.png \
  --output vertical_wave.png --page-warp vertical_wave \
  --intensity 60
```

b. in script

```
transform = WarpTransform(mode='vertical_wave', intensity=60)
save('vertical_wave.png', transform(source))
```

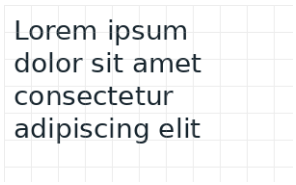
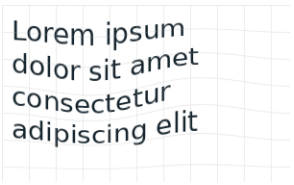
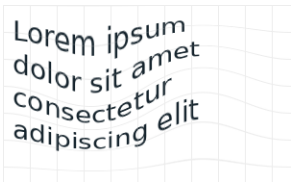
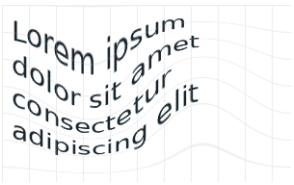
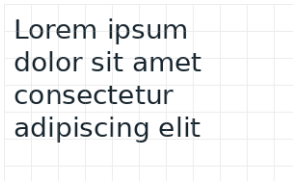
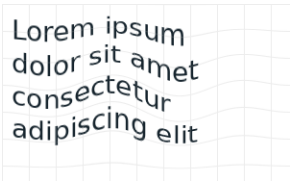
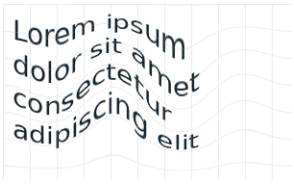
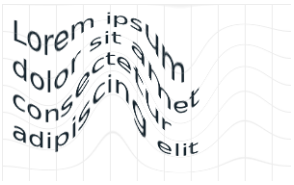
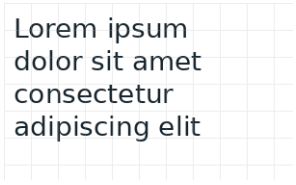
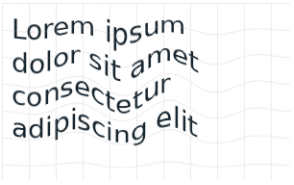
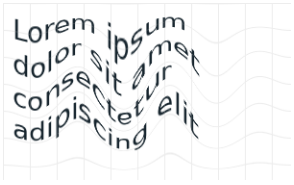
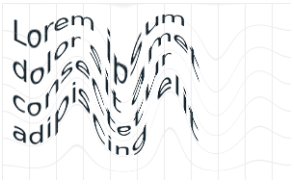
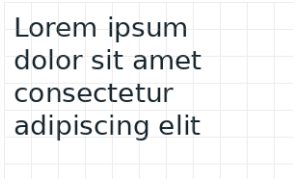
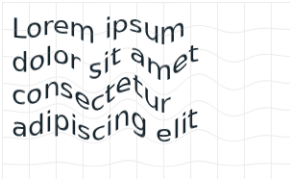
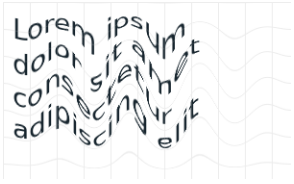
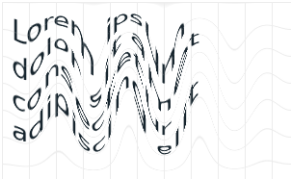
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

9.3 Two-variable legibility study

Rows change vertical wave cycles; columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

Every cell starts with the same dense square reference grid. The transform warps the grid with the text; zero-intensity cells keep straight lines. These lines do not represent droplet centres.

The built-in preset exposes only intensity. This study uses a Python callback with an adjustable wave-cycle count or pinch-envelope power; it is a parameterized variant, not an extra preset CLI flag. The input text itself never changes.

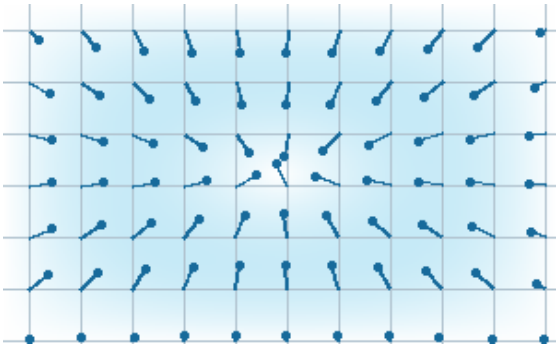
 1 | 0	 1 | 12	 1 | 28	 1 | 50
 2 | 0	 2 | 12	 2 | 28	 2 | 50
 3 | 0	 3 | 12	 3 | 28	 3 | 50
 4 | 0	 4 | 12	 4 | 28	 4 | 50

10. How gentle pinch works

The field points toward the page centre. A smooth envelope holds page edges fixed. The built-in transform exposes pixel intensity, but not a separate width or frequency control.

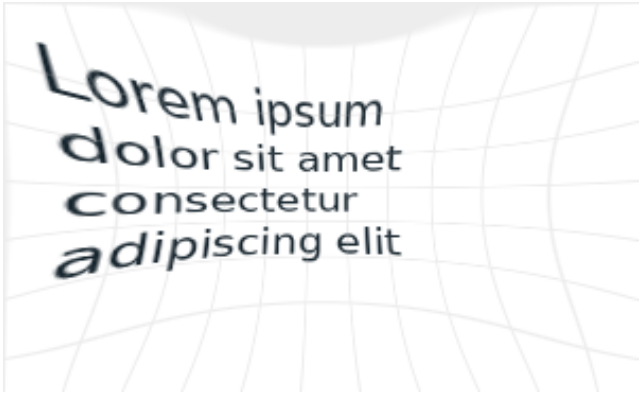


Distorted paragraph



Measured displacement function

The left panel uses intensity 150 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

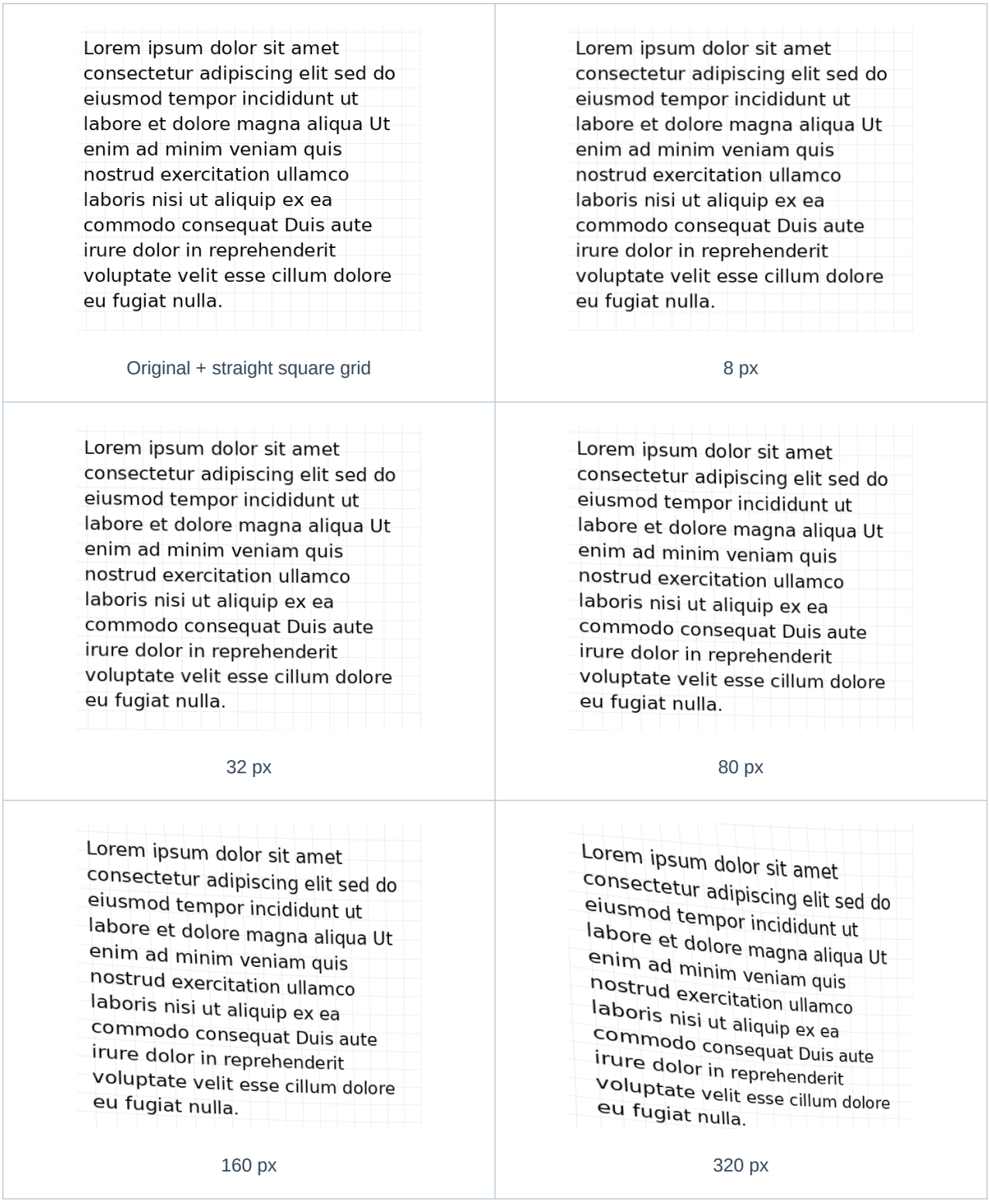


The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

10.1 Gentle pinch: strength examples

A gentle pinch moves text toward the page centre. Its field is small. Use a larger pixel intensity to see the effect.

The first panel shows a straight, dense square grid over the original. Each output uses one gentle_pinch transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



10.2 Run gentle pinch

a. CLI

```
python -m lorem_warped --input original.png \
  --output gentle_pinch.png --page-warp gentle_pinch \
  --intensity 160
```

b. in script

```
transform = WarpTransform(mode='gentle_pinch', intensity=160)
save('gentle_pinch.png', transform(source))
```

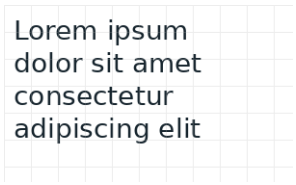
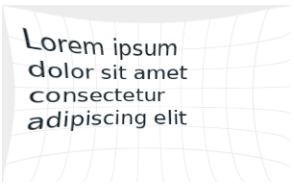
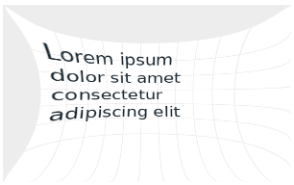
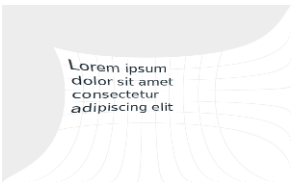
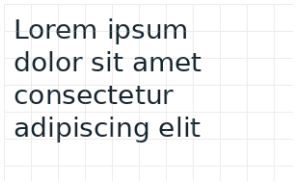
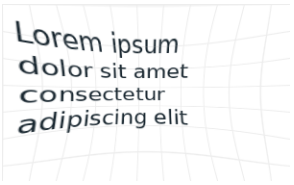
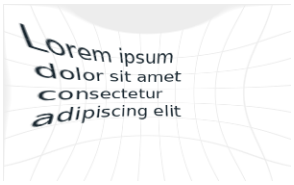
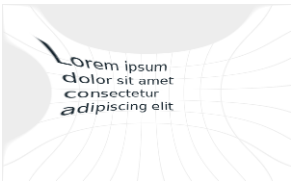
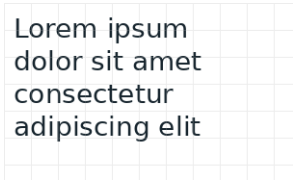
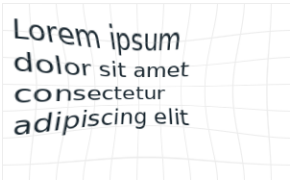
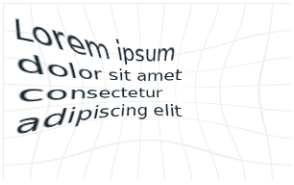
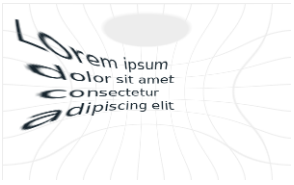
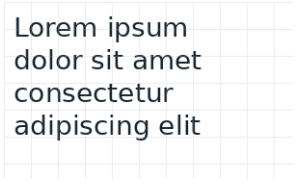
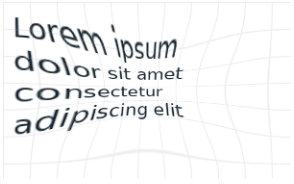
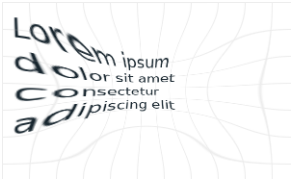
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

10.3 Two-variable legibility study

Rows change pinch envelope power; columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

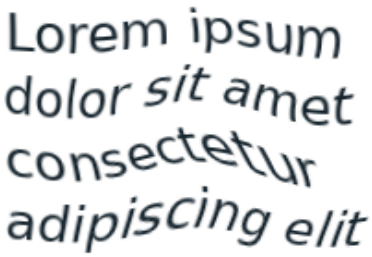
Every cell starts with the same dense square reference grid. The transform warps the grid with the text; zero-intensity cells keep straight lines. These lines do not represent droplet centres.

The built-in preset exposes only intensity. This study uses a Python callback with an adjustable wave-cycle count or pinch-envelope power; it is a parameterized variant, not an extra preset CLI flag. The input text itself never changes.

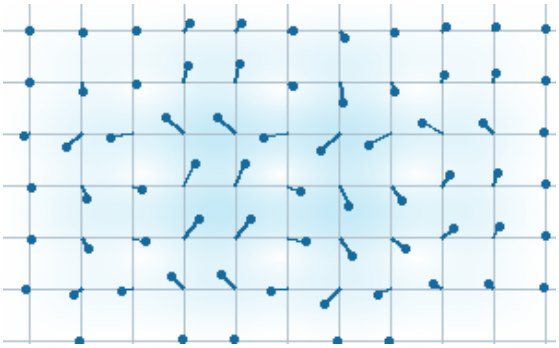
 0.6 | 0	 0.6 | 90	 0.6 | 180	 0.6 | 300
 1 | 0	 1 | 90	 1 | 180	 1 | 300
 1.5 | 0	 1.5 | 90	 1.5 | 180	 1.5 | 300
 2 | 0	 2 | 90	 2 | 180	 2 | 300

11. How custom wave (python only) works

The callback computes independently parameterized X and Y waves from normalized coordinates and returns a (dy, dx) field. Pixel intensity multiplies it. Only the Python API accepts this callback.

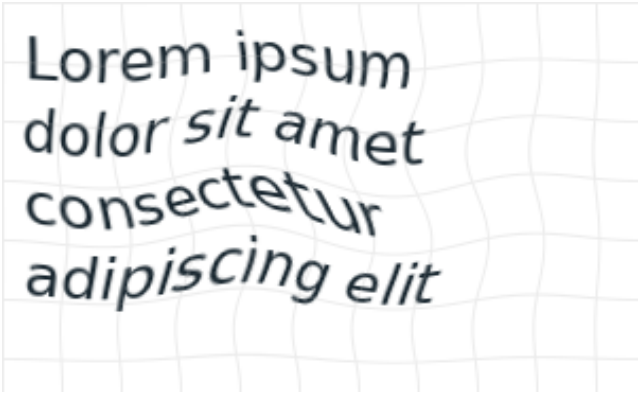


Distorted paragraph



Measured displacement function

The left panel uses X/Y cycles 2/2; intensity 8 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

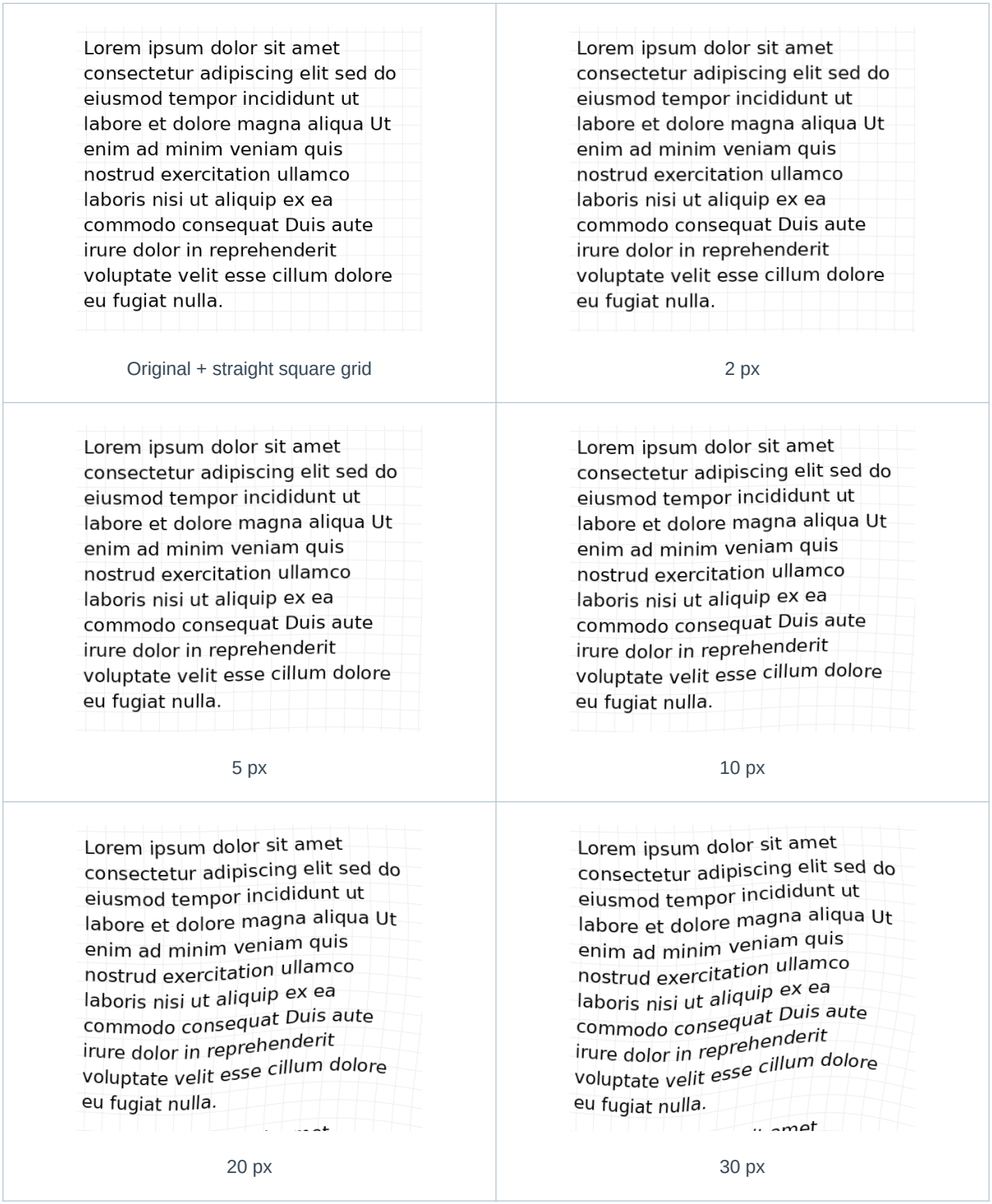


The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

11.1 Custom wave (Python only): strength examples

A custom wave reads normalized Y and X coordinates and returns Y and X displacement tensors. Independent X and Y cycle counts shape the field. The 5, 10, and 20 px examples use a fixed 2-by-2 cycle pattern. The custom mode has no CLI option.

The first panel shows a straight, dense square grid over the original. Each output uses one custom transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



11.2 Run custom wave (python only)

The custom mode has no CLI option. Use the Python API to supply the function below.

a. in script

```
import math
import torch
def custom_wave(cycles_x, cycles_y):
    def field(y, x):
        edge = torch.sin(math.pi * y) * torch.sin(math.pi * x)
        return (edge * torch.sin(2 * math.pi * cycles_x * x),
                edge * torch.sin(2 * math.pi * cycles_y * y))
    return field
transform = WarpTransform(mode='custom', intensity=10,
                          warp_function=custom_wave(2, 2))
save('custom.png', transform(source))
```

In the three 4-by-4 studies, keep intensity fixed at 5, 10, or 20 px, then set cycles_x and cycles_y to 1, 2, 3, or 4. Run each combination on the same source image. No CLI cycle control exists.

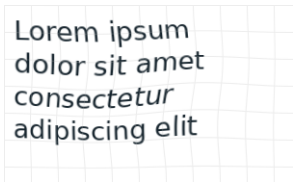
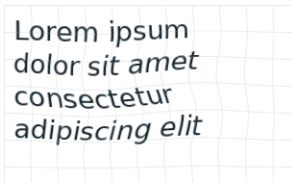
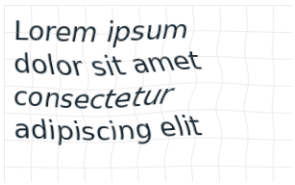
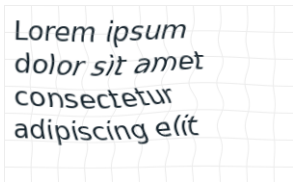
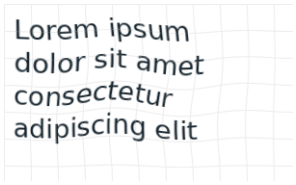
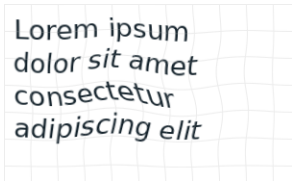
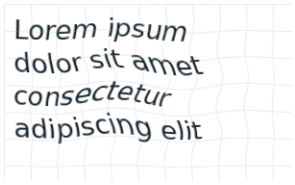
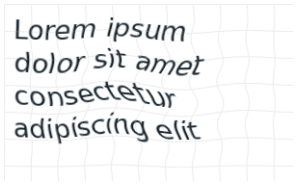
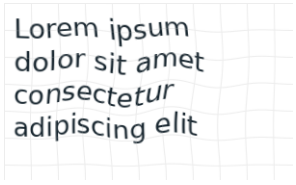
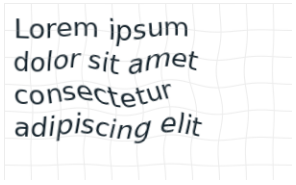
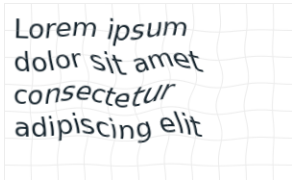
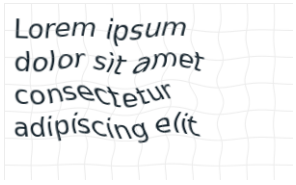
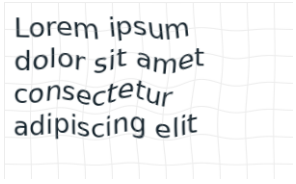
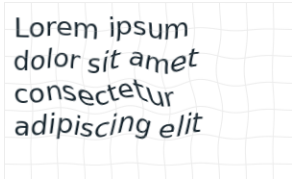
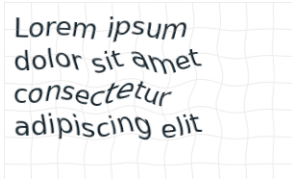
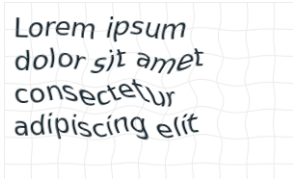
The script writes a PNG. Its callback is Python-only; no equivalent CLI option exists.

11.3 Two-variable legibility study: 5 px

Rows change X cycles; columns change Y cycles. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

The text and straight source grid stay fixed. Each output uses the same callback formula on both text and grid; only its X and Y wave cycles change. Every cell is warped; no zero-intensity control appears on this page.

Keep pixel intensity at 5 throughout this page. X cycles run 1 to 4 down the rows; Y cycles run 1 to 4 across the columns. This is a Python-only callback, not a built-in wave CLI option.

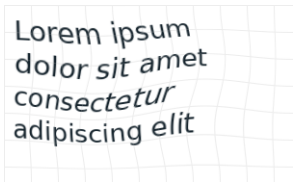
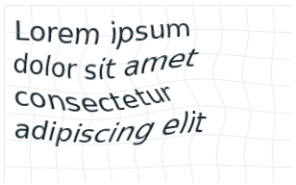
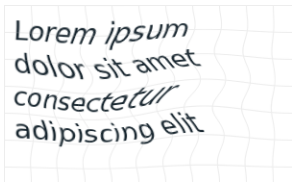
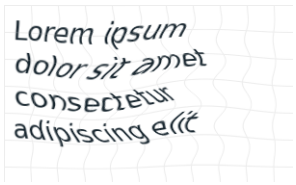
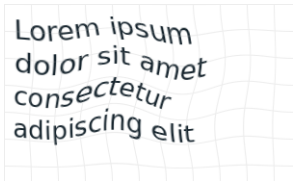
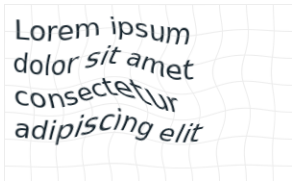
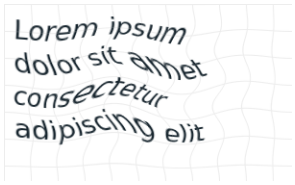
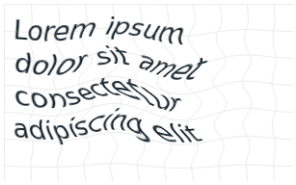
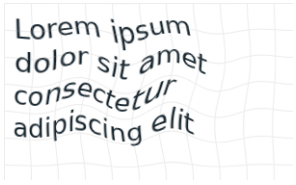
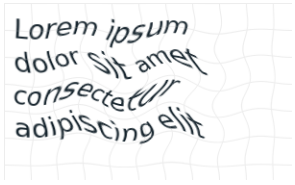
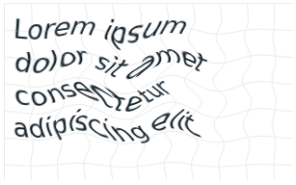
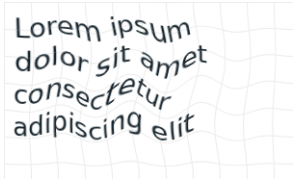
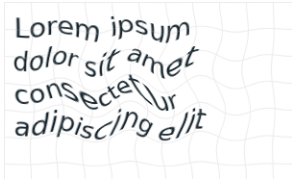
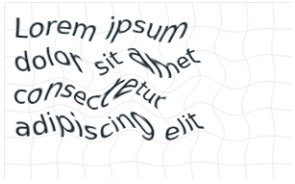
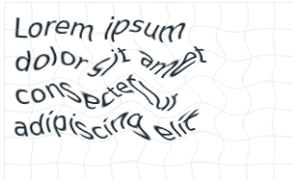
 1 | 1	 1 | 2	 1 | 3	 1 | 4
 2 | 1	 2 | 2	 2 | 3	 2 | 4
 3 | 1	 3 | 2	 3 | 3	 3 | 4
 4 | 1	 4 | 2	 4 | 3	 4 | 4

11.3 Two-variable legibility study: 10 px

Rows change X cycles; columns change Y cycles. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

The text and straight source grid stay fixed. Each output uses the same callback formula on both text and grid; only its X and Y wave cycles change. Every cell is warped; no zero-intensity control appears on this page.

Keep pixel intensity at 10 throughout this page. X cycles run 1 to 4 down the rows; Y cycles run 1 to 4 across the columns. This is a Python-only callback, not a built-in wave CLI option.

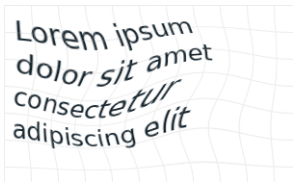
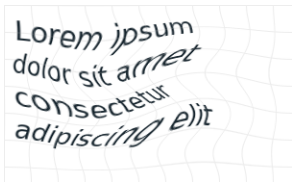
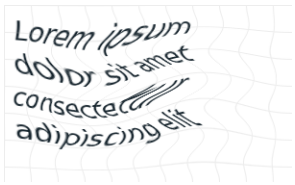
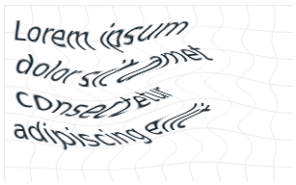
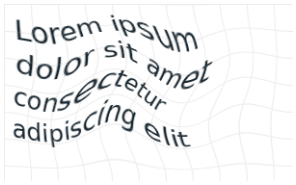
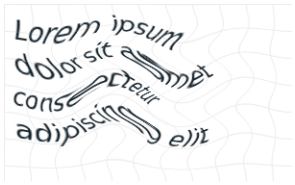
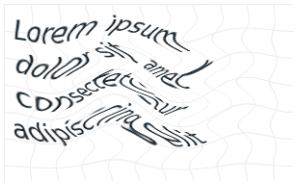
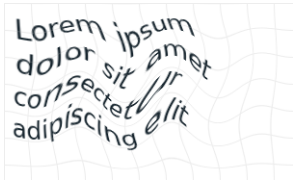
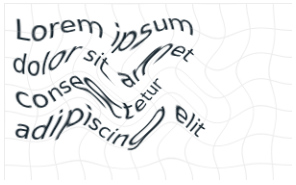
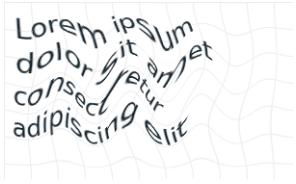
 1 | 1	 1 | 2	 1 | 3	 1 | 4
 2 | 1	 2 | 2	 2 | 3	 2 | 4
 3 | 1	 3 | 2	 3 | 3	 3 | 4
 4 | 1	 4 | 2	 4 | 3	 4 | 4

11.3 Two-variable legibility study: 20 px

Rows change X cycles; columns change Y cycles. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

The text and straight source grid stay fixed. Each output uses the same callback formula on both text and grid; only its X and Y wave cycles change. Every cell is warped; no zero-intensity control appears on this page.

Keep pixel intensity at 20 throughout this page. X cycles run 1 to 4 down the rows; Y cycles run 1 to 4 across the columns. This is a Python-only callback, not a built-in wave CLI option.

 1 | 1	 1 | 2	 1 | 3	 1 | 4
 2 | 1	 2 | 2	 2 | 3	 2 | 4
 3 | 1	 3 | 2	 3 | 3	 3 | 4
 4 | 1	 4 | 2	 4 | 3	 4 | 4

12. Sampling, diagnosis, and references

Geometric fields use pixel displacement. The sampler reads source pixels at output position minus intensity times field. It uses bilinear sampling, border padding, and aligned corners. Strong fields can fold or repeat text.

If the package cannot be imported, activate the correct environment and install it with python -m pip install -e '.[dev]'. For a different physical page size, change both page dimensions in the refractive example.

a. CLI

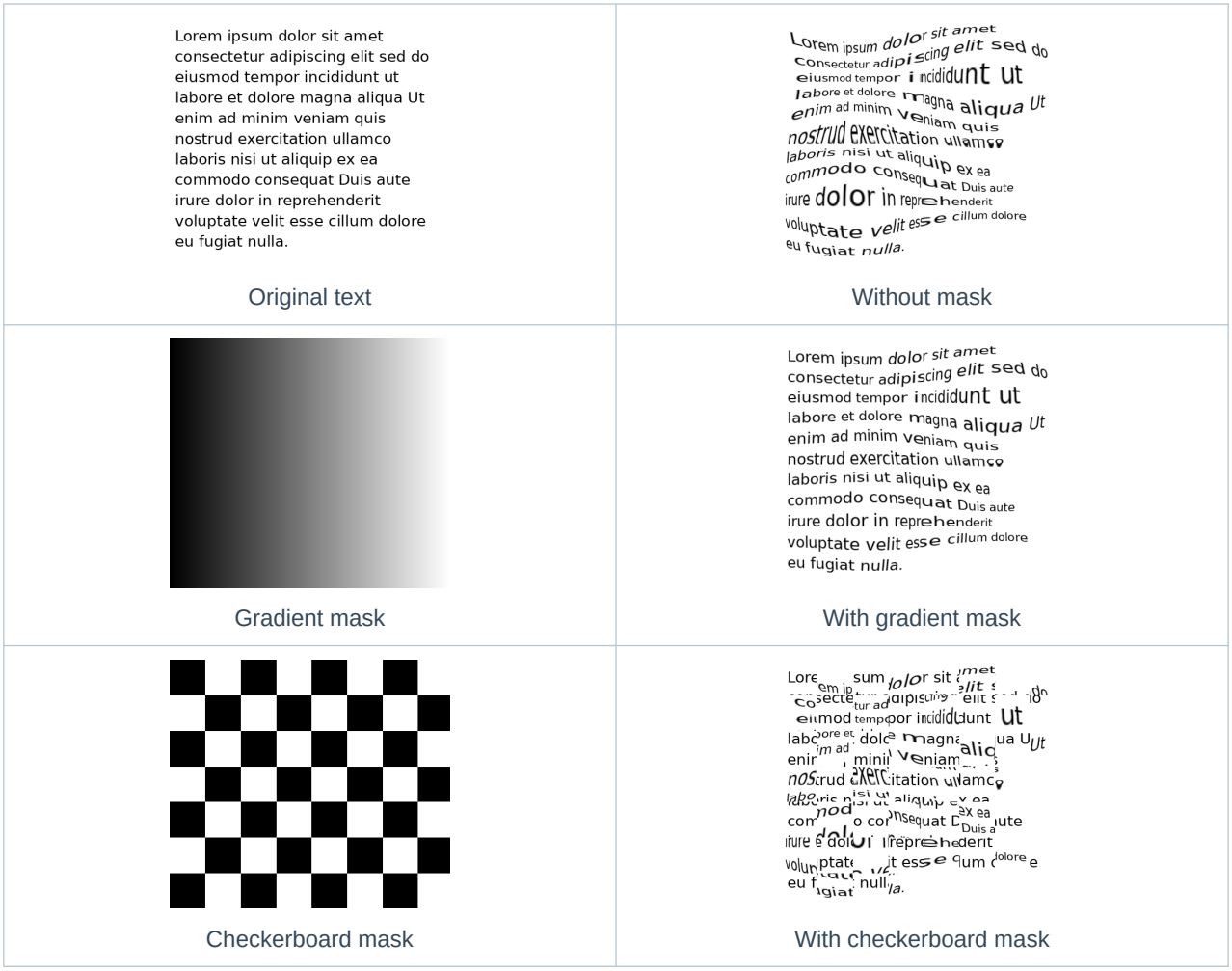
```
python docs/build_manual.py --input original.png \
  --output docs/lorem-warped-manual.pdf
python -m pytest -q
```

Writing reference: ASD-STE100 Issue 9 (January 15, 2025), https://www.asd-ste100.org/assets/files/ASD-STE100_ISSUE9.pdf. Software reference: https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.grid_sample.html. Formal STE conformity is not claimed.

13. Mask the displacement

Use a grayscale mask with exactly the same width and height as the image. At each output pixel, black (0) cancels both displacement components; white (255) keeps the full field. Gray scales both components continuously: 128 gives about half strength. The sampler reads the original at the resulting coordinates. It does not blend an unwarped and a warped image, and it does not change image alpha.

All three results use the same text crop and sine-lens field at 128 px intensity. Row 1 compares the original with an unmasked warp. Row 2 shows a left-to-right gradient mask and its result. Row 3 shows a black-and-white checkerboard mask and its result; each square has side length one-seventh of the crop height in both axes. Only the mask changes. The bundled examples/mask-gradient.png matches the full 1200-by-1600 original.png.



Run this command on the bundled full-size image. To use another input, make a mask of its exact size; the program does not resize the mask. Omit --mask to keep the unmasked behavior.

a. CLI

```
python -m lorem_warped --input original.png \
  --mask examples/mask-gradient.png --page-warp sine_lens \
  --intensity 128 --output masked.png
```

b. in script

```
from pathlib import Path
transform = WarpTransform(mode='sine_lens', intensity=128,
    mask=Path('examples/mask-gradient.png'))
save('masked.png', transform(source))
```

The mask also works with droplets, page/custom warps, and refraction. Refraction retains optical transmission. Python accepts normalized H-by-W tensors; the CLI converts color files to grayscale. Mismatched sizes fail.